UE24CS242B: OPERATING SYSTEMS

Operating Systems Banana Problem

By

NAME: Tanmayi Nagabhairava

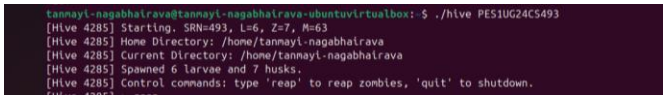
SECTION: I

SRN: PES1UG24CS493

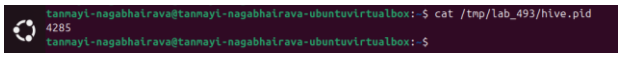
The Hive Containment Protocol

PART A:

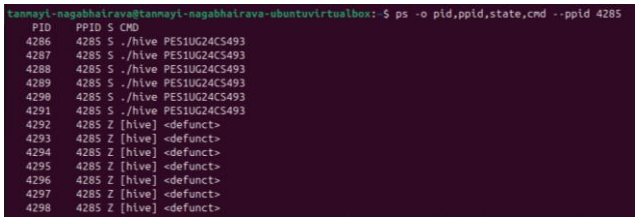
1.



2.



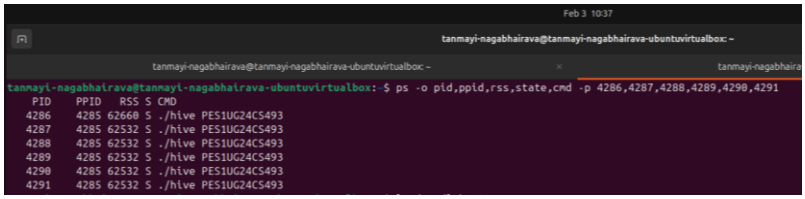
3.



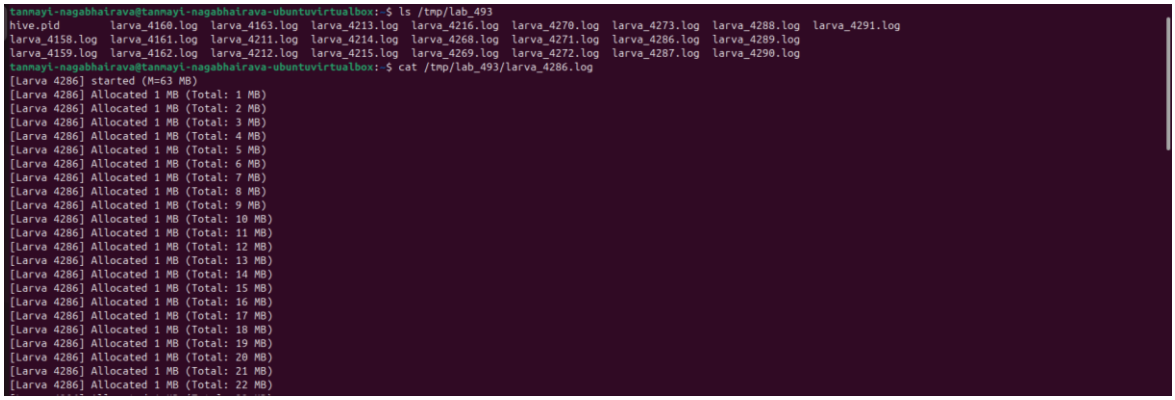
4.



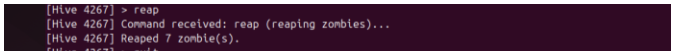
5.



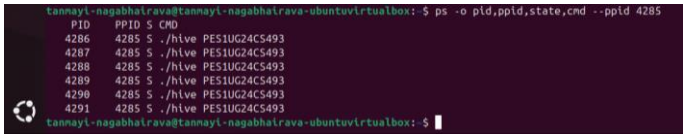
6.



7.



8 and 9.



10. Terminal 1 showing shutdown message)



PART B:

1. Verifying Parent/Child Relationships (PID/PPID)

The Process ID (PID) is a unique number assigned by the operating system to every active process to track it. The Parent Process ID (PPID) is the PID of the specific process that created the child using the fork() system call. You can verify the relationship between the Hive and its children by checking if the PPID of each larva or husk matches the PID of the Hive process. A command like pstree -p visually displays this hierarchy by branching all larvae and husks directly under the Hive parent.

---

2. Parent Responsibility & Zombie Reaping

Zombies exist because the operating system retains a terminated process's exit status in the process table so the parent can read it. The wait() or waitpid() functions are used by the parent to acknowledge this status and "reap" the child, allowing the system to fully clean up the resources. If a parent never reaps its children, the process table entries remain occupied, which can eventually prevent the system from starting new processes. The WNOHANG flag makes the wait non-blocking, allowing the Hive to check for dead children and clean them up without pausing its own execution or becoming unresponsive to user input.

---

3. Zombie Creation

The "Husk" processes are created using fork() and immediately call _exit(), meaning they terminate the moment they are born. They become zombies because the Hive parent is busy waiting for user input and does not automatically call a reaping function after spawning them. The specific parent-side code responsible for this is the main loop, which only triggers the waitpid() reaping logic when the user manually enters the "reap" command.

---

4. Dormant Larvae

A Larva enters a while(1) pause() loop after a memory allocation failure to stay alive in the system instead of exiting. If the larvae simply exited, they would eventually be reaped and disappear from the process table, effectively ending the Hive's impact on the system. By remaining in a paused state, they stay visible in process listings like ps, ensuring they continue to occupy process slots and hold onto their previously allocated memory. This behavior ensures the Hive maintains its presence and continues to strain system resources, fulfilling its "evil plan".